

QueryTriage

How to use it

Five steps: drop the .sql files, open the queue, decide, handle the manual work, rebuild.

1 · Run the pipeline

A folder of .sql files in — a full analysis package out.

Inputs

a directory of .sql files (a scripted schema dump, an SSDT project, a repo checkout) — or a single .sql file

Command

```
python pipeline.py <sql-dir> <outdir>
```

What you get

- *_canonical.json — the normalized object/edge graph
- *_decisions.json — the work queue (stable ids)
- *_dag.md · *_dag.png — layered dependency diagram
- *_review.md · *_hitl.xlsx · *_run_report.json

Parsing

sqlglot builds the AST (confidence=parsed); chunks that fail fall back to regex and are marked inferred — visible, not lost. Credentials are masked before parsing; no db connection needed

2 · Open the review workspace

A local, zero-dependency web client — runs where the data lives.

Start

```
python server.py <outdir>  
-> http://localhost:8787  
same URL for everyone — locally or via container  
(docker run -p 8787:8787; details in step 5)
```

The queue

every warn flag and dropped item is a numbered decision.
Header shows progress: N of M decided — flags ·
assumptions · dropped. Info findings sit below as context.

Click an item

see the canonical source slice and the DAG node it lands on
— cycles show the full A -> B -> ... -> A path

The DAG view

layered topological order with flag tinting — dead code,
cycles and external nodes are visible at a glance

3 · Decide — in verbs, not statuses

The button says what happens to the graph, not what box is ticked.

circular_ref	Break cycle — drop first edge · Keep — document
unreferenced	Entry point — keep · Dead code — drop object
unresolvable_ref	External — accept as-is · Dead link — drop node
write_never_read	Written for external use · Dead table — drop
dynamic_sql	Documented — will map manually
select_star / deprecated_syntax	Rewrite planned
temp_dependency / external_ref	Accept · Confirmed external
not_migrated	Acknowledge — stays dropped

Custom override... always available (whitelisted canonical fields).
ignore on an object or edge strips it before re-analysis — the
DAG is recomputed. If a flag disappears, the decision is marked
stale, not deleted — the audit trail survives.

4 · Manual work — the honest backlog

Everything the engine can't produce itself, counted in the header.

Dynamic SQL

EXEC of variables and sp_executesql of built strings — targets can't be resolved statically; a human reads them

External / missing objects

linked servers, cross-db calls, framework-owned objects — confirm they exist in the target, or cut the dead link

Rewrites

SELECT *, deprecated syntax, temp-table contracts — small, mandatory decisions before any platform move

Inferred edges

regex-fallback extractions marked confidence=inferred — verify them first; they are guesses, not observations

5 · Rebuild — and done

Apply decisions -> rebuild artifacts. ~1 second per cycle.

What rebuilds

dag.png, dag.md, review.md, hitl.xlsx and decisions.json
(merged — your decisions persist)

What changed

ignored objects and edges are gone, cycles visibly break,
stale decisions are marked — the diff is the work

Done means

queue empty AND manual work ~0: a reproducible spec
(canonical + decisions.json) and a DAG you can load from

Prefer Excel?

fill the Decision column in *_hitl.xlsx (ok / reject /
override field=value) and run: decisions.py import <xlsx>

In a container

server.py <outdir> --public binds 0.0.0.0 so Docker can map the
port (docker run -p 8787:8787 -v /data:/data ...) — same URL on
the host. No auth, so: trusted networks only.